L1 L2 ANDROID MASWE-0069 TEST

MASTG-TEST-0334: Native Code Exposed Through WebViews

Overview

This test verifies Android apps that use WebViews with legacy WebView-Native bridges ↗ do not expose native code to websites loaded inside the WebView.

These bridges are created by registering a Java object with the WebView through `addJavascriptInterface` ↗. Public methods of that object that are annotated with `@JavascriptInterface` ↗ become callable from JavaScript running inside the WebView, using the provided `name` as the global JavaScript object.

For this mechanism to work, JavaScript execution must be enabled on the WebView by calling `WebSettings.setJavaScriptEnabled(true)` ↗ (default is `false`), since the exposed interface is invoked from JavaScript code executed within the page.

Steps

1. Use *Reverse Engineering Android Apps* to reverse engineer the app.
2. Use *Static Analysis on Android* to look for references to the relevant WebView APIs.

Observation

The output should contain any references to the relevant WebView APIs.

Evaluation

The test case fails if all the following are true:

- `setJavaScriptEnabled` is explicitly set to `true`.
- `addJavascriptInterface` is used at least once.
- At least one method annotated with `@JavascriptInterface` handles sensitive data or actions and is reachable from untrusted content. See below.

Context Considerations:

To reduce false positives, make sure you understand the context in which the bridge is being used before reporting the associated code as insecure. Ensure that it is being used in a security-relevant context to protect sensitive data or actions, and that it is reachable from untrusted content. For example, if the WebView can load arbitrary or weakly validated URLs, or if the app does not implement proper origin allowlisting for the bridge.

Well-known Challenges when testing for WebView-Native bridges:

- The app may use parametrized or indirect calls to these APIs, for example through utility methods or wrapper classes. Static analysis may not be able to resolve these calls, and dynamic analysis may require specific app states or user interactions to trigger them.
- The app may use several WebViews with different configurations, and it may be difficult to determine which values are set for each WebView instance, especially if they are created dynamically, in different code paths or even across different files.
- The app may use obfuscation, reflection, or dynamic code loading to hide the use of these APIs.

Best Practices

MASTG-BEST-0011: Securely Load File Content in a WebView

MASTG-BEST-0012: Disable JavaScript in WebViews

MASTG-BEST-0013: Disable Content Provider Access in WebViews

MASTG-BEST-0035: Prefer Origin Scoped Messaging Over Legacy JavaScript Bridges

Demos

MASTG-DEMO-0097: Sensitive Data and Functionality Exposed Through WebView JavaScript Bridges